Correlation Power Analysis in AES-128 Algorithm – Part B

7^{H} ΕΡΓΑΣΙΑ

Κωνσταντίνος Μυλωνάς | mli25036 | 03.2026

Πίνακας περιεχομένων

Εισαγωγή 3
Ζήτημα Α: Υπολογισμός Υποθετικής Διαρροής 4
Ερώτηση 1: Πώς υπολογίζετε τη διαρροή αν κάθε υπόθεση κλειδιού ήταν σωστή; 4
Ερώτηση 2: Ποιο είναι το σχήμα του πίνακα υποθετικής διαρροής και γιατί; 4
Ερώτηση 3: Γιατί είναι χρήσιμο να αρχίσετε με λίγα ίχνη για αποσφαλμάτωση; 4
Ερώτηση 4: Γιατί η CPA μπορεί να θεωρηθεί ως στατιστικός έλεγχος υποθέσεων; 4
Ζήτημα Β: CPA – Υπολογισμός Συσχέτισης 6
Ερώτηση 1: Πώς συγκρίνετε την προβλεπόμενη με τη μετρημένη διαρροή; 6
Ερώτηση 2: Πώς υπολογίζεται η συσχέτιση ανά χρονικό δείγμα; 6
Ερώτηση 3: Τι σημαίνει η μέγιστη συσχέτιση για κάθε υπόθεση κλειδιού; 6
Ερώτηση 4: Τι ελέγχους debugging κάνετε αν όλες οι συσχετίσεις είναι επίπεδες; 6
Ζήτημα 3: Επιλογή στόχου AES για CPA επίθεση 9
Ερώτηση 1: Ποιο byte του AES επιλέγετε ως στόχο και γιατί; 9
Ερώτηση 2: Τι ρόλο παίζει το XOR μεταξύ plaintext byte και key byte; 9
Ερώτηση 3: Γιατί η έξοδος S-Box θεωρείται κλασικός στόχος; 9
Ερώτηση 4: Σε ποιον γύρο AES γίνεται η επεξεργασία; 9
Ζήτημα Δ: Υλοποίηση Μοντέλου Διαρροής 12
Ερώτηση 1: Γιατί η μαθηματική ασφάλεια ενός αλγορίθμου δεν συνεπάγεται ασφάλεια της υλοποίησής του; 12
Ερώτηση 2: Ποιες φυσικές διαρροές μπορούν να αξιοποιηθούν από έναν επιτιθέμενο; 12
Ερώτηση 3: Περιγράψτε πώς λειτουργεί καθένα από τα αντίμετρα σε σχέση με τη στατιστική λογική του CPA: 12
Ερώτηση 4: Ποιο είναι το βασικό μειονέκτημα αυτών των τεχνικών; 12
Συμπεράσματα 14

Εισαγωγή

Η Εργασία 7 αποτελεί συνέχεια της Εργασίας 6 και υλοποιεί την πλήρη επίθεση Correlation Power Analysis (CPA) στον AES-128. Χρησιμοποιώντας το μοντέλο Hamming Weight και τα ίχνη κατανάλωσης ισχύος του ATmega328P, εκτελούνται τα υπόλοιπα βήματα της επίθεσης: υπολογισμός υποθετικής διαρροής για όλες τις πιθανές τιμές κλειδιού, υπολογισμός συσχέτισης Pearson μεταξύ προβλεπόμενης και μετρούμενης ισχύος, ανάκτηση του byte κλειδιού και επαλήθευση του αποτελέσματος. Η εργασία ολοκληρώνεται με θεωρητική ανάλυση των αντιμέτρων που προστατεύουν από τέτοιου είδους επιθέσεις.

Ζήτημα Α: Υπολογισμός Υποθετικής Διαρροής

ΕΡΩΤΗΣΗ 1: ΠΩΣ ΥΠΟΛΟΓΙΖΕΤΕ ΤΗ ΔΙΑΡΡΟΗ ΑΝ ΚΑΘΕ ΥΠΟΘΕΣΗ ΚΛΕΙΔΙΟΥ ΗΤΑΝ ΣΩΣΤΗ;

Για κάθε υπόθεση k ∈ {0,...,255} και για κάθε ίχνος i υπολογίζουμε την ενδιάμεση τιμή SBOX[plaintext[i][0] XOR k] και εφαρμόζουμε το μοντέλο Hamming Weight. Το αποτέλεσμα είναι η προβλεπόμενη κατανάλωση ισχύος αν η υπόθεση k ήταν το πραγματικό κλειδί.

ΕΡΩΤΗΣΗ 2: ΠΟΙΟ ΕΙΝΑΙ ΤΟ ΣΧΗΜΑ ΤΟΥ ΠΙΝΑΚΑ ΥΠΟΘΕΤΙΚΗΣ ΔΙΑΡΡΟΗΣ ΚΑΙ ΓΙΑΤΙ;

Ο πίνακας H έχει σχήμα (256, N) — 256 γραμμές για κάθε πιθανή τιμή key byte και N στήλες για κάθε ίχνος. Αυτή η δομή επιτρέπει τη διανυσματική σύγκριση κάθε γραμμής με τα μετρούμενα ίχνη κατά τον υπολογισμό συσχέτισης.

ΕΡΩΤΗΣΗ 3: ΓΙΑΤΙ ΕΙΝΑΙ ΧΡΗΣΙΜΟ ΝΑ ΑΡΧΙΣΕΤΕ ΜΕ ΛΙΓΑ ΙΧΝΗ ΓΙΑ ΑΠΟΣΦΑΛΜΑΤΩΣΗ;

Με λίγα ίχνη (π.χ. 10) ο υπολογισμός είναι άμεσος και μπορούμε να επιθεωρήσουμε χειροκίνητα τις τιμές του πίνακα H και να επαληθεύσουμε ότι το μοντέλο διαρροής λειτουργεί σωστά πριν εκτελέσουμε τον βαρύ υπολογισμό για όλα τα ίχνη και τα 132.296 δείγματα.

ΕΡΩΤΗΣΗ 4: ΓΙΑΤΙ Η CPA ΜΠΟΡΕΙ ΝΑ ΘΕΩΡΗΘΕΙ ΩΣ ΣΤΑΤΙΣΤΙΚΟΣ ΕΛΕΓΧΟΣ ΥΠΟΘΕΣΕΩΝ;

Κάθε τιμή k αποτελεί μια υπόθεση για το πραγματικό κλειδί. Για κάθε υπόθεση υπολογίζουμε τη συσχέτιση μεταξύ προβλεπόμενης και μετρούμενης ισχύος — αυτή είναι η στατιστική της δοκιμής. Η σωστή υπόθεση παράγει στατιστικά σημαντική συσχέτιση, ενώ οι λανθασμένες παράγουν συσχέτιση κοντά στο μηδέν. Η απόφαση λαμβάνεται επιλέγοντας την υπόθεση με τη μέγιστη συσχέτιση — ακριβώς όπως σε έναν στατιστικό έλεγχο επιλέγεται η υπόθεση με τη μεγαλύτερη στατιστική.

```python
# ── Ζήτημα #A – Υπολογισμός Υποθετικής Διαρροής ──────────────
def compute_hypothetical_leakage(plaintexts, byte_idx=0, debug_n=10):
    print("\n" + "–" * 50)
    print("  ΖΗΤΗΜΑ #A – Υπολογισμός Υποθετικής Διαρροής")
    print("–" * 50)

    num_traces = plaintexts.shape[0]

    # Debug με λίγα ίχνη πρώτα
    print(f"\nDebug με {debug_n} ίχνη:")
    H_debug = np.zeros((256, debug_n), dtype=np.uint8)
    for k in range(256):
        for i in range(debug_n):
            intermediate    = SBOX[int(plaintexts[i, byte_idx]) ^ k]
            H_debug[k, i]   = hamming_weight(int(intermediate))
    print(f"H_debug shape : {H_debug.shape}")
    print(f"H_debug[0,:5] : {H_debug[0, :5]}  (υπόθεση k=0x00, πρώτα 5 ίχνη)")
    print(f"H_debug[0x2B,:5]: {H_debug[0x2B, :5]}  (υπόθεση k=0x2B, πρώτα 5 ίχνη)")

    # Πλήρης υπολογισμός – vectorized
    print(f"\nΠλήρης υπολογισμός ({num_traces} ίχνη)...")
    pt_col = plaintexts[:, byte_idx].astype(np.uint16)  # (N,)
    H = np.zeros((256, num_traces), dtype=np.uint8)
    for k in range(256):
        intermediates = SBOX[pt_col ^ k]                # (N,)
        H[k, :]       = hamming_weight(intermediates)

    print(f"H shape       : {H.shape}  (256 υποθέσεις × {num_traces} ίχνη)")
    print(f"H μέσος HW    : {H.mean():.3f}  (αναμενόμενο ≈ 4.0)")

    return H
```

```
————————————————————————————————————————————————————
  ΖΗΤΗΜΑ #A – Υπολογισμός Υποθετικής Διαρροής
————————————————————————————————————————————————————

Debug με 10 ίχνη:
H_debug shape : (256, 10)
H_debug[0,:5] : [4 5 7 4 7]  (υπόθεση k=0x00, πρώτα 5 ίχνη)
H_debug[0x2B,:5]: [6 7 5 2 4]  (υπόθεση k=0x2B, πρώτα 5 ίχνη)

Πλήρης υπολογισμός (1000 ίχνη)...
H shape       : (256, 1000)  (256 υποθέσεις × 1000 ίχνη)
H μέσος HW    : 4.000  (αναμενόμενο ≈ 4.0)
```

Ζήτημα Β: CPA – Υπολογισμός Συσχέτισης

ΕΡΩΤΗΣΗ 1: ΠΩΣ ΣΥΓΚΡΙΝΕΤΕ ΤΗΝ ΠΡΟΒΛΕΠΟΜΕΝΗ ΜΕ ΤΗ ΜΕΤΡΗΜΕΝΗ ΔΙΑΡΡΟΗ;

Υπολογίζουμε τον συντελεστή συσχέτισης Pearson μεταξύ κάθε γραμμής του πίνακα H (προβλεπόμενη ισχύς για υπόθεση k) και κάθε στήλης του πίνακα traces (μετρούμενη ισχύς στο χρονικό δείγμα s). Το αποτέλεσμα R έχει σχήμα (256, S).

ΕΡΩΤΗΣΗ 2: ΠΩΣ ΥΠΟΛΟΓΙΖΕΤΑΙ Η ΣΥΣΧΕΤΙΣΗ ΑΝΑ ΧΡΟΝΙΚΟ ΔΕΙΓΜΑ;

Για κάθε ζεύγος (υπόθεση k, χρονικό δείγμα s) ο συντελεστής Pearson υπολογίζεται ως:

$R[k,s] = \Sigma(H[k,i] - \mu_H) \cdot (T[i,s] - \mu_T) / (\sigma_H \cdot \sigma_T)$

Στην πράξη αυτό γίνεται διανυσματικά με πολλαπλασιασμό πινάκων H_c @ T_c, διαιρούμενο με το γινόμενο των τυπικών αποκλίσεων.

ΕΡΩΤΗΣΗ 3: ΤΙ ΣΗΜΑΙΝΕΙ Η ΜΕΓΙΣΤΗ ΣΥΣΧΕΤΙΣΗ ΓΙΑ ΚΑΘΕ ΥΠΟΘΕΣΗ ΚΛΕΙΔΙΟΥ;

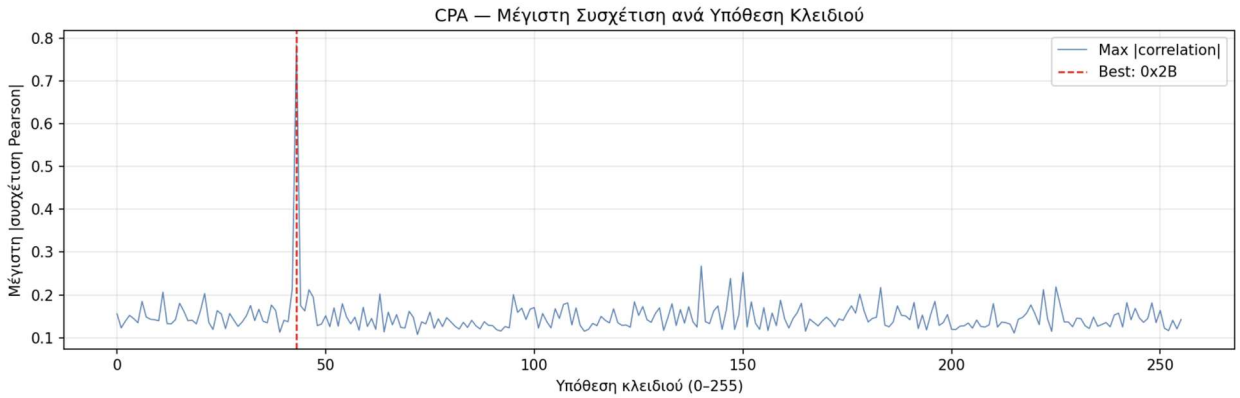
Για κάθε υπόθεση k κρατάμε τη μέγιστη απόλυτη τιμή συσχέτισης σε όλα τα χρονικά δείγματα: $max_corr[k] = \max|R[k,:]|$. Αυτή εκφράζει πόσο καλά η υπόθεση k εξηγεί τη μετρούμενη κατανάλωση ισχύος. Η υπόθεση με τη μέγιστη τιμή είναι το πιο πιθανό σωστό κλειδί.

ΕΡΩΤΗΣΗ 4: ΤΙ ΕΛΕΓΧΟΥΣ DEBUGGING ΚΑΝΕΤΕ ΑΝ ΟΛΕΣ ΟΙ ΣΥΣΧΕΤΙΣΕΙΣ ΕΙΝΑΙ ΕΠΙΠΕΔΕΣ;

Ελέγχουμε διαδοχικά: (α) ότι ο πίνακας H δεν είναι σταθερός — αν όλες οι γραμμές έχουν ίδιες τιμές δεν υπάρχει διακύμανση για συσχέτιση, (β) ότι ο byte_idx αντιστοιχεί στο σωστό byte plaintext, (γ) ότι τα traces και τα plaintexts είναι συγχρονισμένα — ίδια σειρά γραμμών, και (δ) ότι δεν έχει γίνει transpose στους πίνακες H ή traces.

```python
# ─── Ζήτημα #Β – CPA ──────────────────────────────────────────
def compute_cpa(traces, H):
    print("\n" + "-" * 50)
    print("  ΖΗΤΗΜΑ #Β – CPA (Υπολογισμός Συσχέτισης)")
    print("-" * 50)

    num_hyp, num_traces = H.shape
    num_samples         = traces.shape[1]

    # Pearson correlation – vectorized
    # H: (256, N),  traces: (N, S)
    # Αποτέλεσμα R: (256, S)

    H_f = H.astype(np.float32)
    T_f = traces.astype(np.float32)

    H_mean = H_f.mean(axis=1, keepdims=True)          # (256, 1)
    T_mean = T_f.mean(axis=0, keepdims=True)           # (1, S)

    H_c = H_f - H_mean                                 # (256, N)
    T_c = T_f - T_mean                                 # (N, S)

    numerator   = H_c @ T_c                            # (256, S)
    H_std = np.sqrt((H_c ** 2).sum(axis=1, keepdims=True))  # (256, 1)
    T_std = np.sqrt((T_c ** 2).sum(axis=0, keepdims=True))  # (1, S)

    R = numerator / (H_std * T_std + 1e-12)            # (256, S)

    # Μέγιστη απόλυτη συσχέτιση ανά υπόθεση κλειδιού
    max_corr = np.max(np.abs(R), axis=1)               # (256,)

    print(f"R shape       : {R.shape}  (256 υποθέσεις × {num_samples} δείγματα)")
    print(f"Max |R| range : [{max_corr.min():.4f}, {max_corr.max():.4f}]")

    # Γράφημα: max συσχέτιση ανά υπόθεση κλειδιού
    best_k = np.argmax(max_corr)
    plt.figure(figsize=(12, 4))
    plt.plot(max_corr, color='steelblue', linewidth=0.8, label='Max |correlation|')
    plt.axvline(x=best_k, color='red', linestyle='--', linewidth=1.2,
                label=f'Best: 0x{best_k:02X}')
    plt.xlabel('Υπόθεση κλειδιού (0–255)')
    plt.ylabel('Μέγιστη |συσχέτιση Pearson|')
    plt.title('CPA – Μέγιστη Συσχέτιση ανά Υπόθεση Κλειδιού')
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    path = os.path.join(OUTPUT_DIR, "fig1_cpa_max_correlation.png")
    plt.savefig(path, dpi=150)
    plt.close()
    print(f"Αποθηκεύτηκε: {path}")

    return R, max_corr
```

```
────────────────────────────────────────────────────────────
  ΖΗΤΗΜΑ #Β — CPA (Υπολογισμός Συσχέτισης)
────────────────────────────────────────────────────────────
R shape       : (256, 132296)  (256 υποθέσεις × 132296 δείγματα)
Max |R| range : [0.1074, 0.7839]
Αποθηκεύτηκε: figures_project7\fig1_cpa_max_correlation.png
```



Ζήτημα Γ: Επιλογή στόχου AES για CPA επίθεση

ΕΡΩΤΗΣΗ 1: ΠΟΙΟ BYTE ΤΟΥ AES ΕΠΙΛΕΓΕΤΕ ΩΣ ΣΤΟΧΟ ΚΑΙ ΓΙΑΤΙ;

Επιλέγεται το πρώτο byte (byte_idx = 0) της εξόδου του S-Box στον 10 γύρο. Σε πλήρη CPA επίθεση η διαδικασία επαναλαμβάνεται 16 φορές — μία για κάθε byte κλειδιού — για να ανακτηθεί ολόκληρο το 128-bit κλειδί.

ΕΡΩΤΗΣΗ 2: ΤΙ ΡΟΛΟ ΠΑΙΖΕΙ ΤΟ XOR ΜΕΤΑΞΥ PLAINTEXT BYTE ΚΑΙ KEY BYTE;

Το XOR αντιστοιχεί στη λειτουργία AddRoundKey — την πρώτη πράξη του AES. Είναι η μοναδική σύνδεση μεταξύ του γνωστού plaintext και του άγνωστου κλειδιού, και αυτό επιτρέπει τη διατύπωση υποθέσεων: για κάθε πιθανό key_byte (0–255) υπολογίζουμε την ενδιάμεση τιμή και προβλέπουμε την αντίστοιχη κατανάλωση ισχύος.

ΕΡΩΤΗΣΗ 3: ΓΙΑΤΙ Η ΕΞΟΔΟΣ S-BOX ΘΕΩΡΕΙΤΑΙ ΚΛΑΣΙΚΟΣ ΣΤΟΧΟΣ;

Διότι βρίσκεται αμέσως μετά το AddRoundKey, εξαρτάται άμεσα από XOR(plaintext, key), και παράγεται νωρίς στην εκτέλεση πριν η πληροφορία διαχυθεί μέσω ShiftRows και MixColumns. Επιπλέον, η υλοποίηση του S-Box ως lookup table σε 8-bit μικροελεγκτή προκαλεί ισχυρό και αναγνωρίσιμο ενεργειακό αποτύπωμα.

ΕΡΩΤΗΣΗ 4: ΣΕ ΠΟΙΟΝ ΓΥΡΟ AES ΓΙΝΕΤΑΙ Η ΕΠΕΞΕΡΓΑΣΙΑ;

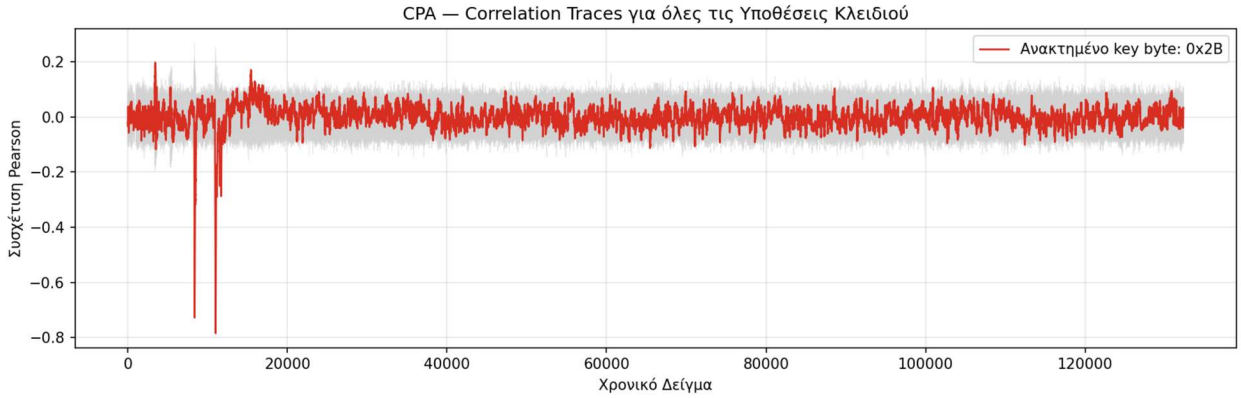
Στον 10 γύρο, στη λειτουργία SubBytes, αμέσως μετά το αρχικό AddRoundKey. Σε αυτό το σημείο η σχέση plaintext–key είναι άμεση και δεν έχει ακόμα διαχυθεί από τις επόμενες λειτουργίες.

```python
# ─── Ζήτημα #Γ – Ανάκτηση & Επαλήθευση Byte Κλειδιού ──────────
def recover_key_byte(R, max_corr, byte_idx=0):
    print("\n" + "-" * 50)
    print("  ΖΗΤΗΜΑ #Γ – Ανάκτηση & Επαλήθευση Byte Κλειδιού")
    print("-" * 50)

    # Ανάκτηση
    best_k         = np.argmax(max_corr)
    best_sample    = np.argmax(np.abs(R[best_k]))
    best_corr_val  = R[best_k, best_sample]

    print(f"Ανακτημένο byte κλειδιού : 0x{best_k:02X}")
    print(f"Χρονικό δείγμα διαρροής  : {best_sample}")
    print(f"Συσχέτιση Pearson        : {best_corr_val:.4f}")

    # Επαλήθευση
    key_hex    = "2B7E151628AED2A6ABF7158809CF4F3C"
    key_bytes  = bytes.fromhex(key_hex)
    expected   = key_bytes[byte_idx]
    result     = "✓ ΣΩΣΤΟ" if best_k == expected else f"✗ ΛΑΘΟΣ (αναμενόταν 0x{expected:02X})"
    print(f"Αναμενόμενο byte         : 0x{expected:02X}")
    print(f"Αποτέλεσμα               : {result}")

    # Γράφημα: correlation trace για το σωστό και λανθασμένα keys
    plt.figure(figsize=(12, 4))
    for k in range(256):
        if k != best_k:
            plt.plot(R[k], color='lightgray', linewidth=0.3, alpha=0.5)
    plt.plot(R[best_k], color='red', linewidth=1.2,
             label=f'Ανακτημένο key byte: 0x{best_k:02X}')
    plt.xlabel('Χρονικό Δείγμα')
    plt.ylabel('Συσχέτιση Pearson')
    plt.title('CPA – Correlation Traces για όλες τις Υποθέσεις Κλειδιού')
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    path = os.path.join(OUTPUT_DIR, "fig2_cpa_correlation_traces.png")
    plt.savefig(path, dpi=150)
    plt.close()
    print(f"Αποθηκεύτηκε: {path}")

    return best_k
```

(function) linewidth: float

```python
# ── Εκτέλεση ──────────────────────────────────────────────
if __name__ == "__main__":
    traces, plaintexts, ciphers = load_dataset('atmega328p.hdf5')

    H              = compute_hypothetical_leakage(plaintexts, byte_idx=0)
    R, max_corr    = compute_cpa(traces, H)
    best_k         = recover_key_byte(R, max_corr, byte_idx=0)

    print("\n" + "—" * 50)
    print(f"  Ολοκλήρωση — γραφήματα στον φάκελο: {OUTPUT_DIR}/")
    print("—" * 50)
```

```
————————————————————————————————————————————————
  ΖΗΤΗΜΑ #Γ — Ανάκτηση & Επαλήθευση Byte Κλειδιού
————————————————————————————————————————————————
Ανακτημένο byte κλειδιού : 0x2B
Χρονικό δείγμα διαρροής  : 11005
Συσχέτιση Pearson        : -0.7839
Αναμενόμενο byte         : 0x2B
Αποτέλεσμα               : ✓ ΣΩΣΤΟ
Αποθηκεύτηκε: figures_project7\fig2_cpa_correlation_traces.png

————————————————————————————————————————————————
  Ολοκλήρωση — γραφήματα στον φάκελο: figures_project7/
————————————————————————————————————————————————
```



Ζήτημα Δ: AES και Αντιμετώπιση Επιθέσεων

ΕΡΩΤΗΣΗ 1: ΓΙΑΤΙ Η ΜΑΘΗΜΑΤΙΚΗ ΑΣΦΑΛΕΙΑ ΕΝΟΣ ΑΛΓΟΡΙΘΜΟΥ ΔΕΝ ΣΥΝΕΠΑΓΕΤΑΙ ΑΣΦΑΛΕΙΑ ΤΗΣ ΥΛΟΠΟΙΗΣΗΣ ΤΟΥ;

Η μαθηματική ασφάλεια αποδεικνύει ότι δεν υπάρχει αποδοτικός αλγόριθμος για την εύρεση του κλειδιού από τα δεδομένα εισόδου/εξόδου. Η φυσική υλοποίηση όμως εισάγει πρόσθετα κανάλια πληροφορίας — κατανάλωση ισχύος, χρόνο εκτέλεσης, ηλεκτρομαγνητικές εκπομπές — που δεν λαμβάνονται υπόψη στο μαθηματικό μοντέλο ασφάλειας. Ο επιτιθέμενος δεν χρειάζεται να «σπάσει» τον αλγόριθμο — αρκεί να παρατηρήσει πώς εκτελείται.

ΕΡΩΤΗΣΗ 2: ΠΟΙΕΣ ΦΥΣΙΚΕΣ ΔΙΑΡΡΟΕΣ ΜΠΟΡΟΥΝ ΝΑ ΑΞΙΟΠΟΙΗΘΟΥΝ ΑΠΟ ΕΝΑΝ ΕΠΙΤΙΘΕΜΕΝΟ;

- Κατανάλωση ισχύος — εξαρτάται από τα επεξεργαζόμενα δεδομένα (SPA, DPA, CPA)
- Ηλεκτρομαγνητικές εκπομπές — ανάλογες της ισχύος, μετρήσιμες από απόσταση
- Χρόνος εκτέλεσης — αν οι λειτουργίες δεν είναι constant-time
- Ακουστικός θόρυβος — από πυκνωτές και μετατροπείς τάσης
- Φαινόμενα σφάλματος (fault injection) — laser, glitching τάσης/ρολογιού

ΕΡΩΤΗΣΗ 3: ΠΕΡΙΓΡΑΨΤΕ ΠΩΣ ΛΕΙΤΟΥΡΓΕΙ ΚΑΘΕΝΑ ΑΠΟ ΤΑ ΑΝΤΙΜΕΤΡΑ ΣΕ ΣΧΕΣΗ ΜΕ ΤΗ ΣΤΑΤΙΣΤΙΚΗ ΛΟΓΙΚΗ ΤΟΥ CPA:

- Masking: Εισάγει τυχαία μάσκα r σε κάθε ενδιάμεση τιμή: v' = v XOR r. Η κατανάλωση ισχύος εξαρτάται πλέον από το v XOR r και όχι μόνο από το v, οπότε η συσχέτιση μεταξύ HW(v) και μετρούμενης ισχύος μηδενίζεται. Η CPA αποτυγχάνει γιατί ο πίνακας H δεν προβλέπει πλέον σωστά την κατανάλωση.
- Shuffling: Εκτελεί τις λειτουργίες του AES σε τυχαία σειρά ανά κρυπτογράφηση. Η διαρροή εξακολουθεί να υπάρχει αλλά εμφανίζεται σε διαφορετικό χρονικό δείγμα κάθε φορά, οπότε η συσχέτιση κατανέμεται σε πολλά δείγματα και αδυνατίζει σημαντικά.
- Προσθήκη θορύβου: Εισάγει τεχνητό τυχαίο θόρυβο στο σήμα ισχύος. Αυξάνει τον αριθμό ιχνών που απαιτούνται για επιτυχή CPA — το SNR μειώνεται και η επίθεση γίνεται πιο δαπανηρή, χωρίς όμως να την καθιστά θεωρητικά αδύνατη.
- Constant-power logic: Σχεδιάζει το κύκλωμα ώστε η κατανάλωση ισχύος να είναι ανεξάρτητη από τα δεδομένα — κάθε λειτουργία καταναλώνει πάντα την ίδια ισχύ. Εξαλείφει πλήρως τη διαρροή στο επίπεδο υλικού, καθιστώντας την CPA θεωρητικά αδύνατη.

ΕΡΩΤΗΣΗ 4: ΠΟΙΟ ΕΙΝΑΙ ΤΟ ΒΑΣΙΚΟ ΜΕΙΟΝΕΚΤΗΜΑ ΑΥΤΩΝ ΤΩΝ ΤΕΧΝΙΚΩΝ;

Κάθε αντίμετρο εισάγει κόστος:

- Masking: Αυξάνει τον αριθμό πράξεων (διπλασιασμός υπολογισμών) και απαιτεί αξιόπιστη πηγή τυχαιότητας.
- Shuffling: Απαιτεί PRNG και επιπλέον λογική διαχείρισης σειράς εκτέλεσης· αυξάνει τον χρόνο εκτέλεσης.

- Προσθήκη θορύβου: Αυξάνει κατανάλωση ισχύος και δεν παρέχει ισχυρή εγγύηση ασφάλειας.
- Constant-power logic: Πολύ υψηλό κόστος υλοποίησης σε hardware· συνήθως 2–4× μεγαλύτερη επιφάνεια κυκλώματος και κατανάλωση ισχύος.

Συμπεράσματα

Η εργασία αυτή υλοποίησε πλήρη επίθεση CPA στον AES-128 και κατέδειξε την επιτυχή ανάκτηση του πρώτου byte κλειδιού (0x2B) από 1.000 ίχνη κατανάλωσης ισχύος του ATmega328P. Η επίθεση δεν εκμεταλλεύτηκε καμία αδυναμία του αλγορίθμου — στηρίχθηκε αποκλειστικά στη στατιστική συσχέτιση μεταξύ προβλεπόμενης και μετρούμενης κατανάλωσης ισχύος.

Το αποτέλεσμα αναδεικνύει το χάσμα μεταξύ θεωρητικής και πρακτικής ασφάλειας: ένας αλγόριθμος που απαιτεί 2^{128} προσπάθειες για brute-force παραβιάζεται με 256 υποθέσεις ανά byte, εφόσον η υλοποίησή του διαρρέει πληροφορία μέσω φυσικών καναλιών. Η ασφάλεια ενός κρυπτογραφικού συστήματος δεν εξαρτάται μόνο από τον αλγόριθμο, αλλά από το σύνολο της υλοποίησης — και αυτό καθιστά τα αντίμετρα όπως masking και constant-power logic εξίσου κρίσιμα με τον ίδιο τον αλγόριθμο.